

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Oltre il codice applicativo

Claude Code per chi gestisce sistemi

Diagnosi, runbook, revisioni: come lo usano i team di Anthropic

- Nel Modulo 4, fin qui, il codice di un'applicazione. Ora i **sistemi e le reti**: log, script, configurazioni, documentazione
- Quando i cluster Kubernetes hanno smesso di schedulare i pod, il team Data Infrastructure di Anthropic ha fatto la **diagnosi** con Claude Code
- Il team Security Engineering gli fa leggere **più fonti** di documentazione, per scrivere runbook e guide di troubleshooting in Markdown
- Per le modifiche all'infrastruttura da approvare, lo stesso team incolla in Claude Code i **piani Terraform**: cicli di feedback più stretti
- Fra ottobre 2025 e aprile 2026, le sessioni di Claude Code per **correggere codice rotto** sono scese dal 33 % al 19 %; quelle per **gestire il software in esercizio** sono salite dal 14 % al 21 %
- **A voce**: qual è il log che aprite più spesso, e che cosa ci cercate?

Un log in una pipe

`claude -p` come un comando qualunque

- In modalità non interattiva, con `-p`, Claude Code **legge lo STDIN**: gli si passano dati in pipe e si reindirizza la risposta, come con qualunque comando. L'esempio della documentazione, un log di build:

```
cat build-error.txt | claude -p \
  'concisely explain the root cause of this build error' \
  > output.txt
```

- Lo STDIN in pipe ha un **tetto di 10 MB**
- `--output-format`: `text`, `json` o `stream-json`, solo con `-p`. In `--allowedTools` l'asterisco finale vale come **prefisso**: `Bash(git diff *)` consente ogni comando che comincia con `git diff`
- Prima di lasciarlo andare da solo: che cosa lo **ferma** (`--max-turns`, `--max-budget-usd`, un limite di spesa), e come sa di aver **finito**

M4-D08 Il log in una pipe

Demo del docente con `claude -p`: lo stesso comando, due ingressi

⚖ Due bracci che differiscono **soltanto nell'ingresso**, **senza strumenti**: le righe di `olt-pergine-01` nel syslog della settimana, oppure il syslog **ripetuto 63 volte**, oltre i 12 MiB

`domanda.txt, la stessa nei due bracci`

```
Questo è il syslog di olt-pergine-01 della settimana: a che ora
inizia e a che ora finisce il guasto di martedì 8 settembre?
Rispondi in una riga: 'inizio HH:MM | fine HH:MM'.
```

```
mkdir -p pipe
p="$(cat domanda.txt)"
for r in filtrato grande; do
  f=olt-pergine-01.log
  if [ "$r" = grande ]; then f=grande.log; fi
  wc -c < "$f" > "pipe/$r.byte"
  cat "$f" | claude -p --model sonnet --effort low --tools "" \
    --strict-mcp-config --no-session-persistence \
    --output-format json -- "$p" > "pipe/$r.out" 2> "pipe/$r.err"
  echo "${PIPESTATUS[1]}" > "pipe/$r.uscita"
done
cat pipe/grande.uscita pipe/grande.err
```

M4-D08 Il log in una pipe

Demo del docente con `claude -p`: lo stesso comando, due ingressi

- Guardate il **codice d'uscita** di `claude`, e che cosa scrive su STDOUT e su STDERR
- Dalla pagina di Claude Code su `claude -p`: «Non-interactive mode reads stdin, so you can pipe data in and redirect the response out like any other command-line tool.» La mette alla prova il braccio **filtrato**, che deve riuscire
- Nella stessa pagina: «Piped stdin is capped at 10MB. If you exceed the cap, Claude Code exits with a clear error and a non-zero status.» Una **garanzia** del prodotto: deve valere a ogni esecuzione
- Il controllo dell'elemento legge i file dei due bracci in `pipe/`: il braccio **grande** deve uscire con un codice diverso da zero, un messaggio d'errore e nessuna risposta, e il tetto si decide solo se il filtrato è riuscito; del **filtrato**, gli orari della risposta si stampano accanto a quelli veri, come **dati**, giusti o sbagliati

M4-D08 Il log in una pipe

Demo del docente con `claude -p`: lo stesso comando, due ingressi

La registrazione del corso (28 ottobre 2026, Claude Code 2.1.312, Sonnet): i due bracci, poi il controllo dell'elemento. Il braccio filtrato:

```
il controllo, sul braccio filtrato: «[…]» segna un taglio

filtrato: 13564 byte in pipe; uscita 0; token in entrata 12308
  risposta: «inizio 10:12 | fine 11:40»
  orari: inizio 10:12 (giusto) · fine 11:40 (giusto)
  la verità, […]: inizio 10:12, fine 11:40
```

- Il braccio grande: «grande: 12664323 byte in pipe; uscita 1; nessun risultato su STDOUT»; su STDERR: «Error: piped stdin input exceeds 10MB. Pass large content as a file path in your prompt instead.»
- Il verdetto: «Lo stdin in pipe ha un tetto di 10 MB: oltre, Claude Code esce con un errore chiaro e un'uscita diversa da zero: **confermata**»
- Il filtro è il punto: si filtra prima, con gli strumenti di sempre, e a Claude arriva **solo ciò che serve**

Il guasto di martedì, dalle sorgenti

Una domanda sui log della rete, e come si controlla la risposta

prompt_senza.txt di M4-D06, senza la delega

```
Sei un tecnico della rete di CiaspoLink e ricostruisci un guasto
dai log degli apparati.

Contesto: martedì 8 settembre 2026 un guasto su olt-pergine-01
ha messo fuori servizio insieme più linee di clienti. I log
della settimana sono nella cartella in cui siamo.

Istruzione: leggi tu i log, con i tuoi strumenti Read, Grep e
Glob.
Poi rispondi: quando è cominciato, quando è finito (quando le
ONT sono tornate in linea) e quante linee di clienti (ONT) ha
coinvolto?

Formato: una riga, 'inizio: HH:MM | fine: HH:MM | ONT: N'.
```

È la domanda della demo della parte «Subagent: delegare o fare da sé»: la sua coppia con la delega a un subagent Explore cambia solo la prima frase dell’istruzione

Il guasto di martedì, dalle sorgenti

Una domanda sui log della rete, e come si controlla la risposta

verita.json

```
{
  […]
  "inizio": "10:12",
  "fine": "11:40",
  "ont": 40,
  […]
}
```

Nella registrazione di M4-D06, sui log della settimana di `progetti/rete`, risposte **giuste**: 3 su 3 con la delega a un subagent Explore (la demo della parte «Subagent: delegare o fare da sé») e 3 su 3 senza. Ogni sessione ha elencato i file della cartella dei log; senza la delega ha risposto dal **syslog** di martedì, mentre i subagent hanno cercato anche negli altri log

Il guasto è anche **altrove**: negli allarmi, «perdita di segnale, 40 ONT offline», poi «segnale ripristinato»; nel PPPoE, «peer is not responding»; nel RADIUS, «Lost-Carrier»; nel DHCP delle linee business, niente. **Incrociare** le sorgenti controlla la risposta

Script con un contratto

La convenzione del corso, sul modello degli script del gestionale

- Lo script che Claude vi scrive diventa un **comando** come gli altri: lo lanciate voi, da cron, in una pipe, da un altro script. Che cosa vuole e come finisce lo dice un **contratto**, scritto in testa
- Il modello: gli script di `bin/` del gestionale, con l'uso e le uscite in testa e gli errori su STDERR. Per esempio, in testa a `bin/inizializza-db.php`, le opzioni `--db`, `--forza` e `--senza-dati`, e «Uscita: 0 database creato; 1 il file esiste già (usa `--forza`); 2 errore.»
- Per gli script che **controllano**, la convenzione del corso, con `--help`:

uscita	quando
0	niente da segnalare
1	almeno una segnalazione: le segnalazioni su STDOUT
2	lo script non può girare (un file che manca, un'opzione sconosciuta): il motivo su STDERR

- È la convenzione **del corso**, non una regola di Anthropic: la chiede la terza traccia del laboratorio. Un contratto così si **prova**: sui dati di esempio, l'uscita che vi aspettate

Per chi gestisce sistemi Claude Code è un collega che legge i log e scrive lo script; il controllo resta vostro.

Configurazioni: generare, poi verificare

Il metodo del corso: uno script che risponde passa o fallisce

- Il metodo del corso per le **configurazioni** degli apparati: farle generare a Claude da un template e dai dati, l'inventario per esempio; poi **verificarle**. Una configurazione che *sembra* giusta non basta
- La verifica è uno **script** che risponde **passa** o **fallisce**, sulle regole che ogni configurazione deve rispettare: le vostre, scritte da voi
- Con un controllo così Claude chiude il ciclo **da solo**: «If you give it something that comes back pass or fail, Claude can close that loop itself.» E va avanti da solo, verificando il proprio lavoro
- Che cosa deve valere lo decidete voi: il controllo **resta vostro**. Un metodo del corso, non una pagina di Anthropic, costruito sul controllo da eseguire che la pagina delle best practice di Claude Code raccomanda

Runbook e documentazione

Dalle fonti sparse a una guida che si segue

- Il team Security Engineering di Anthropic fa leggere a Claude **più fonti** di documentazione, e ne ricava runbook e guide di troubleshooting in Markdown
- La ricetta della documentazione, nella guida dei flussi di lavoro di Claude Code: si parte da ciò che **manca** di documentazione, e si chiude verificando che segua gli **standard del progetto**
- Un esempio sui dati del corso: dal guasto di martedì nei log di `progetti/rete`, un runbook; poi lo segue **chi non l'ha scritto**

Ripetere e sorvegliare

Un comando vostro, un log da seguire, un prompt che si ripete

- Con `!` davanti, un comando di shell va **dritto** al terminale, senza passare da Claude; e Claude risponde al suo output appena compare nella trascrizione
- Il tool **Monitor** esegue un comando in background e passa a Claude **ogni riga** di output: per esempio, seguire un file di log e segnalare gli errori man mano che compaiono
- `/loop` ripete un prompt finché la sessione resta aperta: `/loop 5m check if the deployment finished and tell me what happened`
- Le **scheduled task** nel cloud ci sono anche sul Pro, in **research preview**, sull'infrastruttura di Anthropic: in aula non le usiamo